

CPS2810

Computer Organization and Architecture

Unit 1 Review

Bart Gajderowicz
bart.gajderowicz@usu.edu
SER 301F

Unit 1 Topics

- We will cover:

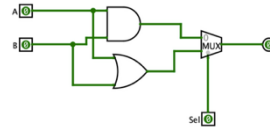
- Introductions: me and course



- Boolean Algebra

$\text{True} \wedge \text{False} = \text{False}$

- Introduction to chips



- Logisim Fundamentals



Course Objectives

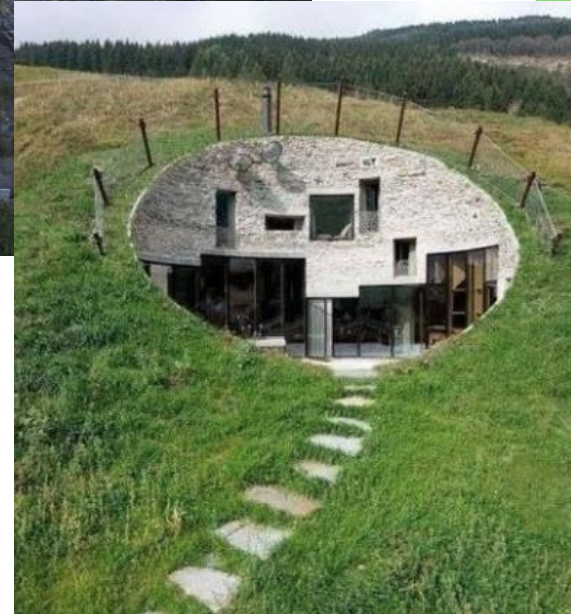
- Think like a Computer Architect
- Eight Great Ideas
- Binary Numbers and Arithmetic
- Boolean Algebra, logic gates, Logisim
- Combinatorial logic, decoders, multiplexors
- 2-level logic and PLAs, arrays of logic elements
- 32-bit ALU
- Clocks, flip-flops, latches, and registers
- Hardware for arithmetic: multiply and divide
- Floating point numbers, ASCII (end of material for e1)
- MIPS: instruction set architecture, adding, subtracting, registers
- The processor
- Pipelining
- Virtual memory

What is Computer Architecture?

Architecture

- Appreciate the importance of out-of-the-box and creative thinking
- Think about the tradeoffs in the design of the building
 - Strength, weaknesses
- Derive principles on your own for good design and innovation.

Find the differences: This and That



Question: How to design a house?

- What to consider?
 - Higher costs, surroundings, etc
- Depends on what you want?
- How is a home designed?
 - Hard work, perseverance, dedication (over decades)
 - Experience
 - Creativity
 - Out-of-the-box thinking
 - Principled design
 - A good understanding of past designs
 - Good judgment and intuition
 - String combination of skills (math, architecture, art, etc)

High-Level Goals

- Understand the principles of computer architecture
- Understand the precedents
- Understand tradeoffs of different designs and ideas.
- Implement principled design.
- Understand novel, out-of-the-box designs.
- Principles, precedents, and how to use them for new designs.

Role of the (Computer) Architect

- Look Backwards (Examine old code)
 - Understand tradeoffs and designs, upsides/downsides, past workloads.
 - Analyze and evaluate the past.
- Look Forward (listen to the dreamers)
 - Be the dreamer and create new designs. Listen to dreamers.
 - Push the state of the arts. Evaluate new design choices.
- Look Up (nature of the problems)
 - Understand important problems and their nature.
 - Develop architectures and ideas to solve important problems.
- Look Down (predict the future of technology)
 - Understand the capabilities of the underlying technology.
 - Predict and adapt to the future of technology. Enable the future of technology.

Takeaways

- Being an (computer) architect is not easy
- You need to consider many things in designing a new system and have good intuition/insight into ideas/tradeoffs.
 - What is the workflow?
 - What are the processes?
- But it is fun and can be very technically rewarding.
- Enables a great future
 - E.g. many scientific and everyday-life innovations would not have been possible without architectural innovations that enabled very high-performance systems
- This course will teach you how to be a better programmer.

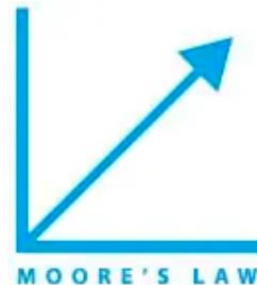
Eight Great Ideas in Computer Architecture

Overview

- We will explore the Eight Great Ideas in Computer Architecture.
- They guide the design and evolution of modern computer systems.

1. Design for Moore's Law

- Hardware improvements are expected over time, so designs anticipate faster, cheaper devices.
- The one constant for computer designers is rapid change, which is driven largely by Moore's Law.
- It states that integrated circuit resources double every 18–24 months.
 - Moore's Law resulted from a 1965 prediction of such growth in IC capacity made by Gordon Moore, one of the founders of Intel.
 - As computer designs can take years, the resources available per chip can easily double or quadruple between the start and finish of the project.
 - Like a skeet shooter, computer architects must anticipate where the technology will be when the design finishes rather than design for where it starts.



2. Use Abstraction to Simplify Design

- Break complex systems into layers and modules for easier understanding and development.



3. Make the Common Case Fast

- Optimize the most frequently used operations for better overall performance.



4. Performance via Parallelism

- Use multiple processors or cores to work on tasks simultaneously.



5. Performance via Pipelining

- Divide tasks into stages so multiple instructions can be executed in an assembly line fashion.



6. Performance via Prediction

- Speculate on future operations (e.g., branch prediction) to reduce delays.



7. Hierarchy of Memories

- Organize memory into layers (cache, RAM, disk) for speed and cost efficiency.



8. Dependability via Redundancy

- Use backup components to ensure reliability and fault tolerance.



Welcome to CS2810!

Bart Gajderowicz

bart.gajderowicz@usu.edu

bartg.org

Any Questions?



Source: <http://what-if.xkcd.com/14/>

Unit 1 Topics

- We will cover:

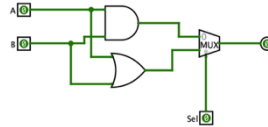
- Introductions



- **Boolean Algebra**

True • False = False

- Introduction to chips



- Logisim Fundamentals



1.2 Boolean Algebra

How do computers represent information?



on high true 1



off low false 0

we call these bits



George Boole
1815-1864

1.2 Boolean Algebra

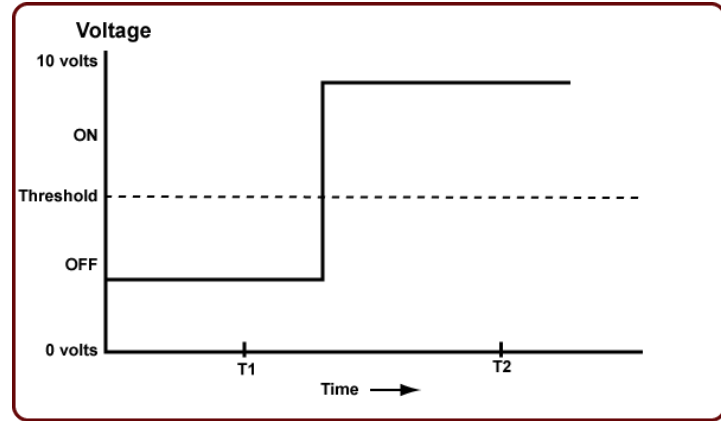
How do computers REALLY represent information?



on high true 1



off low false 0



Data 1 1 0 0 1 0 1 0 1 1 0 0 1 0 1 1

unipolar



Sending data

How to store information with only 2 values?

b_2 b_1 b_0



1 binary variable (1 bit) = 2 possible states (2^1)

2 binary variables (2 bits) = 4 possible states (2^2)

3 binary variables (3 bits) = 8 possible states (2^3)

Boolean Operators

How can we manipulate boolean values to perform complex operations?

- Addition, subtraction, storing information, and display pixels to a screen!!

Boolean operators

- A function that inputs and outputs boolean values

Three fundamental boolean operators

- NOT operator
- AND operator
- OR operator

**The combination of these three operators
lets you build any computer!**

Truth Table: NOT Operator

$$f(x) = \text{NOT } x$$

x	f(x)
	
	

Truth Table: NOT Operator

$$f(x) = \text{NOT } x$$

x	f(x)
0	1
1	0

Truth Table

Question: Can you drive a car?

- x = You failed your driving test.
- y = You can drive a car.

- $y = !x$

shorthand for
not(x)

- success: if $x=0$, you did not fail your test, and you can drive.

Truth Table: AND Operator

$$f(x, y) = x \text{ AND } y$$

x	y	f(x, y)
0	0	0
0	1	0
1	0	0
1	1	1

Question: Can you log in?

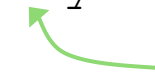
- x = You entered the correct username.
- y = You entered the correct password.
- $f(x, y) = x \cdot y$
 shorthand for $\text{AND}(x, y)$
- success: if $x=1$ and $y=1$, you can log in

Truth Table: OR Operator

$$f(x, y) = x \text{ OR } y$$

x	y	f(x, y)
0	0	0
0	1	1
1	0	1
1	1	1

Question: Are you ready for rain?

- x = You have an umbrella.
- y = You have a raincoat.
- $f(x, y) = x + y$
shorthand for $\text{OR}(x, y)$
- success: if $x=1$ or $y=1$, you are ready for rain

Real-Life Boolean Algebra Expression

- Similar to regular algebra, we can construct an equation of boolean values.
 - $A = \text{I need to go to school}$
 - $B = \text{Weather is nice}$
 - $C = \text{Bike is working}$
- I will walk to school $= A \bullet (B + !C)$
- I will walk to school **IF**:
 - I need to go to school **AND** either (the weather is nice **OR** bike is not working)

Question: Multiple Choice

Which of the following is equivalent to $(A + B) \bullet (B + !A)$

- **A:** $B + (A \bullet !A)$
- **B:** B
- **C:** $B \bullet (A + A)$
- **D:** A

Question: Multiple Choice

Which of the following is equivalent to $!(B + !A)$

- **A:** B
- **B:** $!B + !A$
- **C:** $B + A$
- **D:** $!B \bullet A$

Fundamental Laws of Boolean Algebra

Boolean Laws

$$A \bullet 1 = ?$$

$$A \bullet 1 = A$$

Identity Law

Booleans Laws (write these down)

Name	AND Form	OR Form
Identity Law	$1 \bullet A = A$	$0 + A = A$
Zero's and One's Law	$0 \bullet A = 0$	$1 + A = 1$
Inverse Law	$A \bullet !A = 0$	$A + !A = 1$
Commutative Law	$A \bullet B = B \bullet A$	$A + B = B + A$
Associative Law	$A \bullet (B \bullet C) = (A \bullet B) \bullet C$	$A + (B + C) = (A + B) + C$
Distributive Law	$A \bullet (B + C) = (A \bullet B) + (A \bullet C)$	$A + (B \bullet C) = (A + B) \bullet (A + C)$
De Morgen's Law	$!(A \bullet B) = !A + !B$	$!(A + C) = !A \bullet !C$

1) Identity Law

- A variable remains unchanged when OR-ed with 0 and AND-ed with 1
- Example:
 - $A + 0 = A$
 - $A \cdot 1 = A$

2) Zero and One Law

- Zero is the identity for OR, and one is the identity for AND.
- Examples:
 - $0 \bullet A = 0$
 - $1 + A = 1$

3) Inverse Law

- A variable ANDed with its negation results in 0, and OR-ed with its negation results in 1.
- Examples:
 - $A \cdot !A = 0$
 - $A + !A = 1$

4) Commutative Law

- Order of operands does not matter for AND and OR.
- Example:
 - $A + B = B + A$
 - $A \cdot B = B \cdot A$

5) Associative Law

- Grouping of operands does not affect the result for AND and OR.
- Example:
 - $(A + B) + C = A + (B + C)$
 - $(A \cdot B) \cdot C = A \cdot (B \cdot C)$

6) Distributive Law

- Distributes an operation over another operation.
- Example:
 - $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
 - $A + (B \cdot C) = (A + B) \cdot (A + C)$

7) De Morgan's Law

- Describes how to distribute NOT across AND and OR operations.
- The trickiest law
 - $A = \text{It is raining}$
 - $B = \text{It is cold}$
- $!(A + B)$
 - It is not true that it is raining or it is cold
 - Essentially saying that neither can be true
- $!A \bullet !B$
 - This is the same as saying it is not raining AND it is not cold.

$$!(A \bullet B) = !A + !B$$

$$!(A + B) = !A \bullet !B$$

Notice that it flips NOT, as well as the AND and OR operators.

Can flip a NOT expression; convert an AND expression into an OR expression, and vice versa.

Simplifying Complex Boolean Algebra Expressions

1) $\boxed{!(A \bullet B)} \bullet (A + !B)$

2) $(!A \boxed{+ !B}) \bullet (A \boxed{+ !B})$

3) $!B + \boxed{(!A \bullet A)}$

4) $!B \boxed{+ 0}$

5) $!B$

$\therefore !(A \bullet B) \bullet (A + !B) = !B$

(De Morgen's Law) (put ! inside)

(Distributive Law) (move !B outside)

(Inverse Law) (simplify tautology)

(Identity Law) (remove +0)

Conclusion: Expression (1) is not dependent on A; just the inverse of B:

if B=0 then Expression (1) = 1 if B=1 then Expression (1)=0

Simplifying Complex Boolean Algebra Expressions

1) $(A + B) + (!B + !A)$

2) $(A + B + !B + !A)$

3) $(A + !A + B + !B)$

4) $(A + !A) + (B + !B)$

5) $1 + 1$

6) 1

$\therefore (A + B) + (!B + !A) = 1$

(Associative Law) (combine expressions)

(Commutative Law) (same vars together)

(Associative Law) (group same vars)

(Inverse Law) (resolve tautologies)

(Simplify) (remove redundant values)

Conclusion: Expression (1) is always 1, regardless of A or B

Question: Simplify

1. $!(!(A \bullet !B) + !(B \bullet !A)) \bullet (!A + !B)$

Question: Simplify

1. $!(!(A \bullet !B) + !(B \bullet !A)) \bullet (!A + !B)$

2. $((A \bullet !B) \bullet (B \bullet !A)) \bullet (!A + !B)$

(De Morgen's Law)

3. $(A \bullet !B \bullet B \bullet !A) \bullet (!A + !B)$

(Associative Law)

4. $(A \bullet !A \bullet B \bullet !B) \bullet (!A + !B)$

(Commutative Law)

5. $0 \bullet (!A + !B)$

(Inverse Law)

6. 0

(Zero's Law)

Unit 1 Topics

- We will cover:

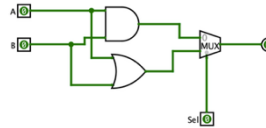
- Introductions



- Boolean Algebra

True • False = False

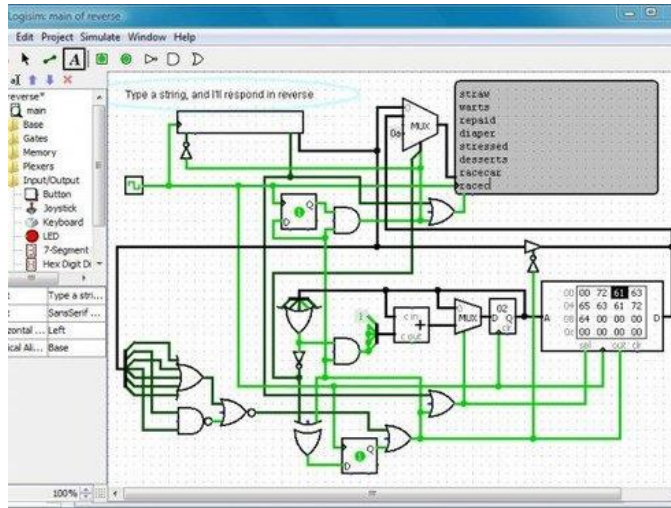
- Introduction to logic gates



- Logisim Fundamentals



Logisim Evolution



- For your Assignments in this course, you will be using the Logisim Evolution software to design, build, and simulate your logic gates.
- Logisim installation instructions are part of **Assignment 0**.
- In **Unit 1.4** on Canvas, you will find videos that go over the Logisim Evolution software

1.3 Introduction to Logic Gates

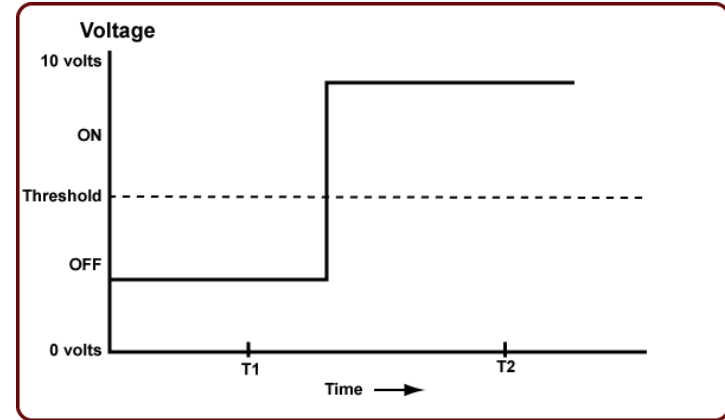
Binary representation and algebra are not just for fun.



on high true 1

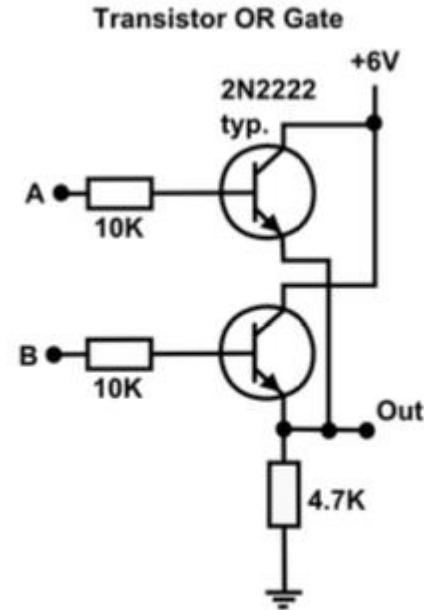


off low false 0



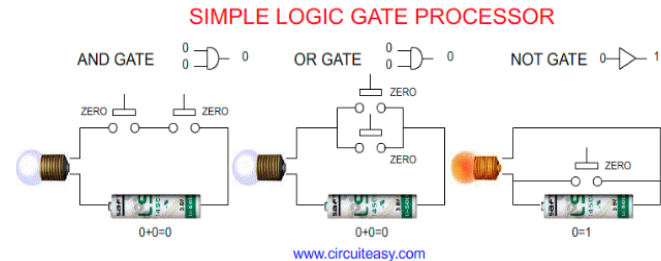
Why?

- All computers can be built by combining AND/OR/NOT operations
- These operations are implemented using physical hardware (transistors)
- More simplified operations
=
Lower cost + more efficient system

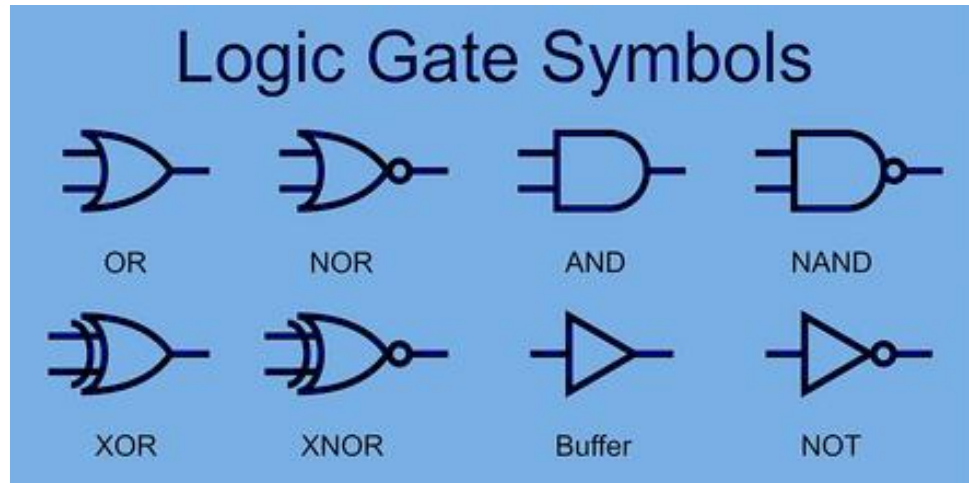


Logic Gates

- Physical realization of boolean algebra
- Building blocks for modern computers
- And/Or/Not gates, as well as many more

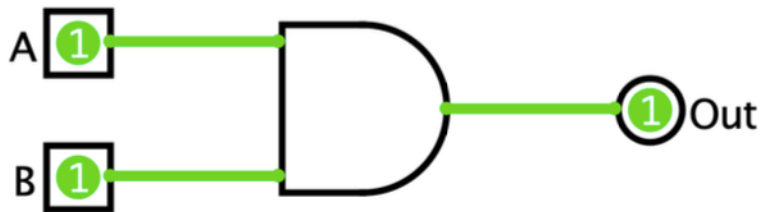


Logic Gate Types



- Some familiar ones:
 - AND/OR/NOT
- Some new ones:
 - NOR
 - NAND
 - XOR
 - XNOR
 - Buffer

AND Gate

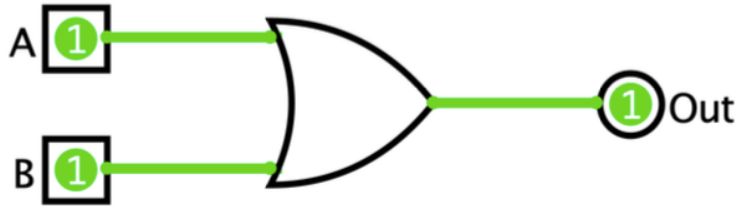


AND Gate

Implements the $\bullet$ operators: e.g. $A \text{ AND } B$

A	B	AND(A,B)
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate



OR Gate

Implements the + operator: e.g. $A \text{ OR } B$

A	B	OR(A,B)
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

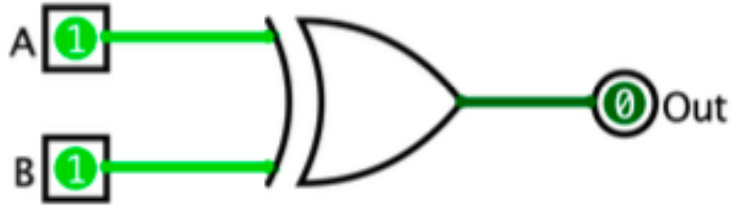


A	NOT(A)
0	1
1	0

NOT Gate

Implements the ! operator: e.g. NOT(A)

XOR Gate

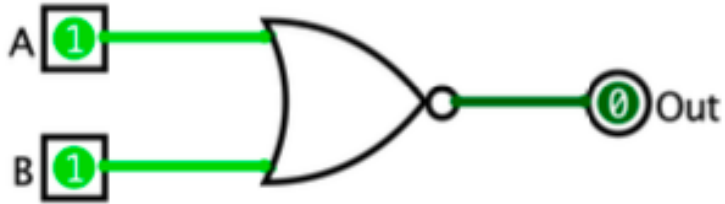


Exclusive OR Gate

Outputs true if and only if ONE of the inputs are true, but not both

a	b	XOR(a,b)
0	0	0
0	1	1
1	0	1
1	1	0

NOR Gate

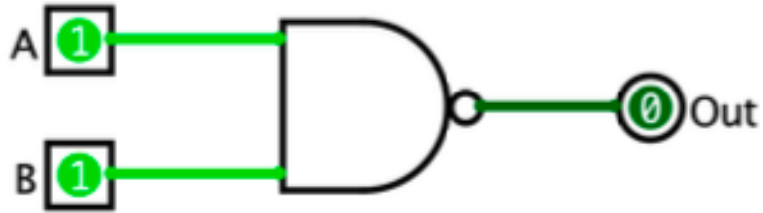


NOT OR

Logical complement of OR, i.e. $\text{NOT}(a \text{ OR } b)$

a	b	NOR(a,b)
0	0	1
0	1	0
1	0	0
1	1	0

NAND Gate

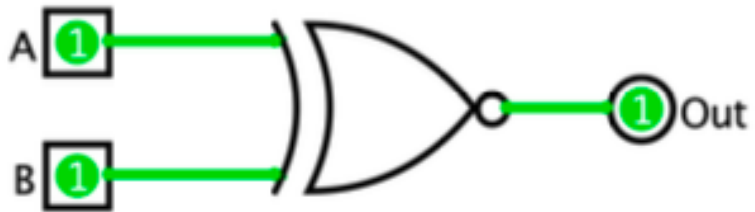


a	b	NAND(a,b)
0	0	1
0	1	1
1	0	1
1	1	0

NOT AND

Logical complement of AND, i.e. $\text{NOT}(a \text{ AND } b)$

XNOR Gate



NOT XOR

Logical complement of XOR, i.e. $\text{NOT}(a \text{ XOR } b)$

a	b	XNOR(a,b)
0	0	1
0	1	0
1	0	0
1	1	1

Representing Arbitrary Functions

No.	x	y	z	f(x,y,z)
1)	0	0	0	0
2)	0	0	1	1
3)	0	1	0	0
4)	0	1	1	0
5)	1	0	0	1
6)	1	0	1	0
7)	1	1	0	0
8)	1	1	1	1

$\neg x \bullet \neg y \bullet z$

OR

$x \bullet \neg y \bullet \neg z$

OR

$x \bullet y \bullet z$

Converting Truth Table to Boolean Expression

1. Only focus on the rows with an output $f(x,y,z)$ of 1

Rows: 2, 5, 8

2. Convert each row to a series of ANDs and NOTs

$a == 0 \rightarrow \neg a$

$a == 1 \rightarrow a$

3. Join each intermediate expression with an OR

Final Expression

$(\neg x \bullet \neg y \bullet z) + (x \bullet \neg y \bullet \neg z) + (x \bullet y \bullet z)$

Question: Logic Gates

- Draw the Logic Gates for one of the ORed expressions

1. $(!x \bullet !y \bullet z)$

2. $(x \bullet !y \bullet !z)$

3. $(x \bullet y \bullet z)$

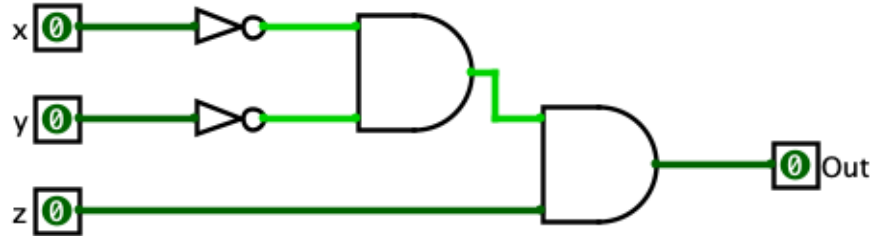
Question: Logic Gates

- Draw the Logic Gates for one of the ORed expressions

1. $(!x \bullet !y \bullet z)$

2. $(x \bullet !y \bullet !z)$

3. $(x \bullet y \bullet z)$



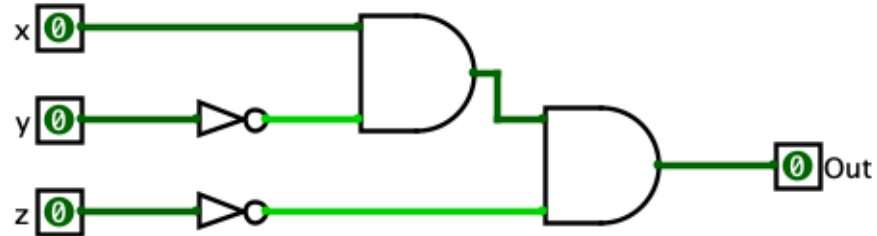
Question: Logic Gates

- Draw the Logic Gates for one of the ORed expressions

1. $(!x \bullet !y \bullet z)$

2. $(x \bullet !y \bullet !z)$

3. $(x \bullet y \bullet z)$



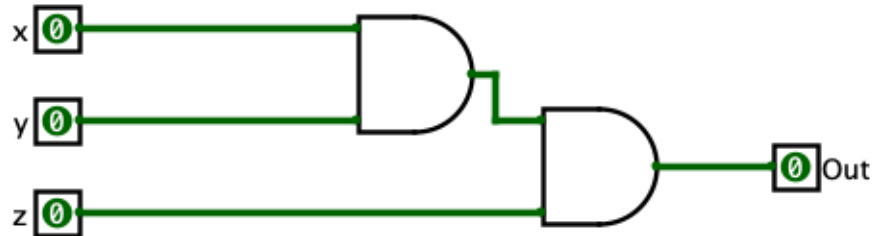
Question: Logic Gates

- Draw the Logic Gates for one of the ORed expressions

1. $(!x \bullet !y \bullet z)$

2. $(x \bullet !y \bullet !z)$

3. $(x \bullet y \bullet z)$



Question – Create Truth Table

- Create the Truth Table for the following expression:

$$(!x \bullet y) + (x \bullet !y \bullet z)$$

Hint: create two columns for each ORed expression, and identify where they are (1).

Then combine them with OR.

x	y	z	!x	!y	(!x • y)	(x • !y • z)	f(x,y,z)
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	1	0	1	0	1	0	1
0	1	1	1	0	1	0	1
1	0	0	0	1	0	0	0
1	0	1	0	1	0	1	1
1	1	0	0	0	0	0	0
1	1	1	0	0	0	0	0

NAND – the Universal Gate

x	y	NAND
0	0	1
0	1	1
1	0	1
1	1	0

x	NOT
0	1
1	0

x	y	AND
0	0	0
0	1	0
1	0	0
1	1	1

x	y	OR
0	0	0
0	1	1
1	0	1
1	1	1

Observations

- $\text{NOT}(x) = x \text{ NAND } x$
- $\text{AND}(x,y) = \text{NOT}(x \text{ NAND } y)$
- $\text{OR}(x,y) = \text{NOT}(\text{NOT}(x \text{ OR } y))$
 $= \text{NOT}(\text{NOT}(x) \text{ AND } \text{NOT}(y))$
 $= (x \text{ NAND } x) \text{ NAND } (y \text{ NAND } y)$

Theorems

- NOT can be realized using NAND
- AND can be realized using NAND
- OR can be realized using NAND

Conclusion

- Boolean algebra is used to form complex relationships between boolean values.
- We can use laws to manipulate and simplify boolean relationships.
- Logic Gates allow us to realize boolean expressions on hardware.
- All functions can be realized using elementary logic gates.
 - All functions can be built out of only NAND gates!

CS2810

Any Questions?



Source: <http://what-if.xkcd.com/14/>

Bart Gajderowicz

bart.gajderowicz@usu.edu

bartg.org